

• PLANNER'S MASTER PLAN / GROUND TRUTH VERIFIED FROM CODE

SentioBot: from working demo to production-grade, evaluated, deployed

A sequenced plan that holds one discipline above all: **measure before you optimize, ship honesty before you ship features**. Every phase ends in something you can put on a resume and defend under scrutiny.

Stack: **FastAPI** + **LangGraph** + **Next.js** + **Supabase**

Retrieval: **ChromaDB** hybrid

LLM: **Gemini 2.0 Flash**

Recon date: **2026-07-17**

01 What this project actually is today

The seeded plan and the README describe an older single-file Streamlit app. The code on disk tells a very different, more advanced story. This is the corrected picture the whole plan is built on.



Backend

FastAPI service (v2.0.0) with JWT auth, SSE token streaming at `/chat/stream`, and a **LangGraph** **StateGraph** agent, not the old ReAct executor.



Retrieval

ChromaDB + BM25 ensemble (0.4 / 0.6), parent-document store, wrapped in a `MultiQueryRetriever`. Chroma is still live; pgvector is only a commented-out note.



Frontend

Next.js 14 + React + Tailwind. Real streaming chat, tool-call badges, sidebar, markdown, feedback buttons. Already sophisticated, not Streamlit.



Data

Supabase Postgres for users, orders, products, conversations, messages, analytics, tickets. It is the app DB, not the vector store.



Cache

Three tiers: in-process LRU, semantic cosine (0.92), optional Redis. Checked before the agent runs on every request.



Deploy scaffold

docker-compose (backend + frontend + redis), railway.toml, vercel.json. Present but not yet proven to boot (see defects).

README / SEED SAYS

- Single **Streamlit** app, `app.py` + `dashboard.py`
- LangChain **ReAct AgentExecutor**, Gemini **1.5** Flash
- **ChromaDB** replaced by **Supabase/pgvector**?
- Setup via `conda` + `pip freeze`
- Feedback logged to `analytics.log`, works

CODE ACTUALLY DOES

- Two services: **FastAPI** + **Next.js**, no Streamlit anywhere
- **LangGraph** StateGraph, Gemini **2.0** Flash, temp 0.1
- Chroma **stays**; Supabase is the app DB only
- Pinned backend/`requirements.txt` already exists
- Feedback UI renders but is **wired wrong and never posts**

02

Defects found during recon

Read from source, not assumed. Severity drives when each gets fixed. The headline: the service most likely does not boot as configured, so that is fixed first, before any measurement.

P0

Boot integrity: the app likely does not start as configured

Contradictory imports (`cache.py` uses `from config` while the rest use `backend.*`) plus a `Dockerfile/railway uvicorn main:app` that cannot resolve after the build flattens the folder. No single documented command boots it.

P0

Cross-user cache leakage (privacy)

The cache ignores `user_id` by design, but answers embed the user's name, serial numbers and warranty state. Alice's personalized answer can be served to Bob at cosine ≥ 0.92 .

P1

Agent tool path has no RAG context

Keyword routing sends "warranty/order" questions to a graph whose prompt says "RAG first" but binds no lookup tool and injects no documents, contradicting "answer only from context".

P1

Feedback broken end to end

The frontend never captures X-Interaction-Id, so `interactionId` is always undefined and thumbs up/down silently no-op. Analytics feedback stays near zero.

P1

Tool-path answer overwrite + empty sources

`full_answer = str(output)` clobbers streamed content with raw tool text, and tool turns always send `sources: []`. Persisted messages can be malformed.

P2

Frontend build config missing

No `tailwind.config.js` / `postcss.config.js` / `tsconfig.json` in the listing, so a fresh `npm run build` likely ships unstyled or errors. Confirm, then fix.

P2

Insecure defaults + latency blind spots

Hardcoded default `jwt_secret`, placeholder CORS origin, and only a total-ms logger: no TTFT, no retrieval timing, no p50/p95 on a streaming app.

03

The phased roadmap

Eight phases, each with an explicit gate (what "done" means) and a resume artifact. Nothing shiny jumps the line: features are last, and no optimization starts before its baseline is committed.

Why this order (the learning)

An "improvement" with no frozen baseline is a story, not a result. You cannot claim "40% faster" or "faithfulness 0.9" unless a committed, reproducible before-number exists. So we make it run, freeze baselines for latency and quality, only then optimize and re-measure, then harden, then deploy, and rewrite the README last so it describes what truly exists.

P0

Boot integrity + repo honesty

ACTIVE: INCREMENT 1

One canonical run config that boots locally and in Docker. Fix the import scheme and the container command. No logic changes. Secrets scan. Trim the worst stale README lines.

GATE Fresh clone installs, one command boots, `/health` is ok, and one real chat streams a token + done event, in the container too.

Artifact: **clone, install, one command, it answers** + a clean secrets-scan line.

P1

Baseline: latency

MEASURE

Instrument the current system with no changes: time-to-first-token, retrieval time (including the hidden multi-query LLM call), and end-to-end p50/p95, for both the RAG and tool paths. Drive it through the real endpoint.

GATE TTFT + retrieval + e2e p50/p95 committed, stamped with commit/model/date, reproduced twice.

Artifact: **a real latency table with methodology**.

P2

Baseline: quality

RATIFY + FREEZE

A ~50-item golden set (doc lookup, orders, warranty, tickets, multi-turn, must-refuse), ratified then frozen. Metrics pinned before results: hit-rate@k, context precision, RAGAS faithfulness + relevancy (2 runs for variance), tool-call correctness.

GATE Baseline scores committed and reproduced twice; golden set frozen; every score carries its recipe.

Artifact: **a reproducible quality number** to replace the deleted "70%" claim.

P3

Measured optimization

OPTIMIZE

Only the ranked candidates below, each with its baseline already committed. Every change re-runs both baselines on the same frozen inputs; deltas reported honestly, including regressions.

GATE Each change shows a measured delta on the frozen set; no candidate starts before its baseline exists.

Artifact: **a before/after table** ("cut p95 TTFT 40%, faithfulness held within noise").

P4

Service hardening

DEFEND

Fix the recon defects on the real path (cache leak, tool-path context, feedback, answer overwrite, secrets/CORS). Timeouts, retries with backoff, defined failure messages, cost guards, a real rate limiter, injection resistance.

GATE The reviewer's hostile battery (100K input, injection, network kill, out-of-scope) cannot crash or leak.

Artifact: **"hardened for production" that is demonstrable.**

P5

Container + CI

AUTOMATE

Proper multi-stage Docker, ratified index strategy (bake vs mount). GitHub Actions: lint, unit tests, frontend build, and a free eval smoke. No unbounded paid LLM calls in CI, ever.

GATE docker compose up from scratch serves both services; CI is green.

Artifact: **CI badge + "containerized, CI-gated".**

P6

Deploy + observability + honest README

SHIP

Backend on Railway or Cloud Run (scale-to-zero), frontend on Vercel, secrets in the platform manager. Per-request tracing and cost-per-query. README rewritten last to match reality: diagram, eval + latency tables, live link, honest limitations.

GATE A public URL answers a real question; every README claim is true of the code at that commit.

Artifact: **a live demo link + a fully verifiable resume bullet.**

P7

Net-new features

GROW

The full feature backlog, gated behind baselines + hardening. Each gets its own planner / builder / reviewer loop and is re-measured so it cannot regress P1/P2.

GATE Each feature ships behind its own baseline; none regresses latency or quality.

Backlog, roughly in order:



Mobile + accessible UI

Responsive drawer sidebar, a11y, keyboard nav, PWA. This is the "simple to sophisticated UI" ask, as polish, since it is already React.



Citation highlighting

Extract and show the exact supporting sentence in the sources panel.



UX polish

Stop / regenerate, copy button, skeleton states, smoother streaming.



Admin dashboard

Real analytics page, now that feedback actually writes.



User knowledge bases

Multi-document upload so users bring their own docs.



Multimodal RAG

Ingest diagrams/images from manuals via a vision model.

Brainstormed as a critic, not a cheerleader. Each states the expected win, the measurement that would prove it, the cost, and the risk. None is approved until its baseline is committed. Rank = build order inside P3.

1 Cut or gate the MultiQueryRetriever

med risk

- WIN Biggest TTFT drop available; removes an LLM round-trip and halves per-query LLM cost.
- PROOF P1 TTFT delta; P2 hit-rate@k + faithfulness must hold.
- COST Engineering only; reduces recurring spend.
- RISK Multi-query may be lifting recall; keep only if quality holds.

2 Fix, then exploit, the cache

low risk

- WIN Near-zero latency on repeats, safely, once scoped per user.
- PROOF Hit-rate + TTFT-on-hit vs miss; reviewer confirms no leak.
- COST Engineering only.
- RISK 0.92 semantic threshold can over-fire; measure honestly.

3 Model routing (small model for easy queries)

med risk

- WIN Lower latency + cost by reserving the strong model for tool reasoning.
- PROOF Per-class TTFT + cost delta, quality held per class.
- COST Engineering; net cost down.
- RISK Router must be cheap (not another LLM call) or the win vanishes.

4 Unify retrieval into the tool path

med risk

- WIN Correctness (context on tool turns) + one honest eval path. Also fixes P1-3.
- PROOF P2 quality on tool-routed questions rises; tool-path TTFT measured.
- COST Engineering.
- RISK Adds retrieval latency to tool turns; overlaps P4, sequence care.

5 Warm singletons + share the embedder

low risk

- WIN Kills first-request cold start; drops a duplicate MiniLM instance.
- PROOF First-request TTFT and steady-state RSS before/after.
- COST Small engineering.
- RISK Slower boot, more idle RAM; watch on scale-to-zero.

6

Index / chunking design pass

high risk

- WIN Better hit-rate@k and/or a smaller, faster index.
- PROOF P2 retrieval metrics + index size/retrieve time.
- COST Engineering + one reingest.
- RISK Tuning against the golden set is leakage; needs a held-out split first.

7

Ensemble weight sweep

low risk

- WIN Modest hit-rate gain over the current 0.4 / 0.6.
- PROOF Retrieval metrics across a small grid on a held-out split.
- COST Cheap and deterministic, no LLM judge.
- RISK Leakage if fit on the eval set; low technical risk.

P7

pgvector migration

high effort

- WIN One datastore, simpler deploy, maybe faster at scale.
- PROOF Quality parity on the frozen set + retrieve-time at this corpus.
- COST Engineering, possibly Supabase compute.
- RISK Unclear win at 85 docs. A feature-grade migration, not an optimization.

05

How each increment gets built

Three separate Claude sessions, one relay loop. The planner never edits code; the reviewer never shares the builder's context. That separation is the point: it is what catches the leaked secret, the unreproducible number, the defect the builder rationalized.

**Planner**

Cowork, read-only

Reads the repo, ratifies findings, ranks candidates, writes the spec + gate into STATUS-prod.md. Holds the line on order and evidence. (This document.)

**Builder**

Claude Code

Inspects the real code, builds one increment to the ratified spec, verifies against the gate, commits. Asks before every push, never commits secrets.

Reviewer

fresh Claude Code

Trusts nothing. Re-runs the tests, the eval, the secrets scan itself. Tries to break the change. Writes a verdict; P0/P1 findings go back before the next increment.

The cadence, every increment

Inspect the substrate → planner ratifies and writes the kickoff → builder builds and verifies the gate → reviewer tears it apart → log commits and move to the next. The frozen golden set and the committed baselines are what make every later claim defensible.

SentioBot production-grade workstream · planner recon 2026-07-17 · source of truth: STATUS-prod.md in the repo. This roadmap mirrors that file; the file is authoritative.